5. USING COOKIES, SESSION AND SERVER OBJECT

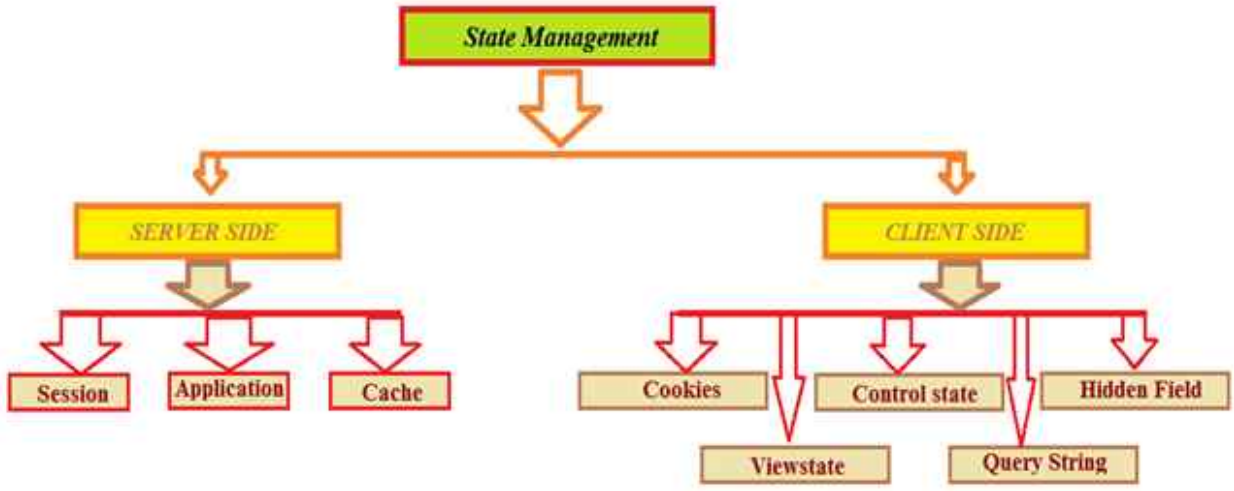
5.1 Cookies Definition, Advantages and Disadvantages of Cookies

What are Cookies?

- Cookies are the small files that are **created on the client's system** or client browser memory (if temporary).
- It is used for **state management** .
- We can store small pieces of information in a client system and use it when needed. The most interesting thing is that it works transparently with the user.
- It can be easily used anywhere in your web application.
- Cookies store information in plain text format. If a web application uses cookies, the server sends cookies and the client browser will store it. The browser then returns the cookie to the server the next time the page is requested. The most common examples of using a cookie are to store user information, user preferences, password remember option, etc.

This file is located on client machines "C:\Document and Settings\Currently_Login user\Cookie" path.

For example, a cookie may contain information on the last video the user watched or what items they have in their shopping cart. Many e-commerce websites like Amazon save cookies to your computer that can either collect your data or store the items in the shopping cart.



How are Cookies started?

When a client requests to the server, the server sends cookies to the client. The same cookies can be referred to for subsequent requests. For example, if *codeproject.com* stores the session ID as cookies, when a client hits the first time on the server, the server generates the session ID and sends it as a cookie to the client [as shown in Fig. 1.0.].

Now for all subsequent requests from the same client, it uses the session-ID from the cookies, just like in the picture below:

The browser and web server are responsible for exchanging cookies information. For different sites, the browser keeps cookies differently. If a page needs information from the cookies, when that URL is hit, first it searches the local system for cookies information, then it is moved to the server with that information.





Advantages of Cookies

Following are the main advantages of using cookies in a web application:

- It's very simple to use and implement.
- Its clear text so user can able to read it.
- Browser takes care of sending the data.

- For multiple sites with cookies, the browser automatically arranges them.
- Its easy way to maintain.
- Fast accessing.

Disadvantages of Cookies

The main disadvantages of cookies are:

- It stores data in simple text format, so it's not secure at all.
- There is a size limit for cookies data (4096 bytes / 4KB).
- The maximum number of cookies allowed is also limited. Most browsers provide limits the number of cookies to 20. If new cookies come, the old ones are discarded. Some browsers support up to 300.
- We need to configure the browser. Cookies will not work on a high security configuration of the browser.

5.2 Creating a Cookies, Removing Cookies

For working with cookies, we need to use the namespace System.web.

Creating/Writing Cookies

There are many ways to create cookies, I am going to outline some of them below:

Way 1 (by using HttpCookies class)

```C#
//First Way
HttpCookie StudentCookies = new HttpCookie("StudentCookies");
StudentCookies.Value = TextBox1.Text;
StudentCookies.Expires = DateTime.Now.AddHours(1);
Response.Cookies.Add(StudentCookies);
```

Way 2 (by using Response directly)

```C#
//Second Way
Response.Cookies["StudentCookies"].Value = TextBox1.Text;
Response.Cookies["StudentCookies"].Expires = DateTime.Now.AddDays(1);
```

Way 3 (multiple values in same cookie)

C# Copy Code

```
//Writing Multiple values in single cookie
Response.Cookies["StudentCookies"]["RollNumber"] = TextBox1.Text;
Response.Cookies["StudentCookies"]["FirstName"] = "Abhimanyu";
Response.Cookies["StudentCookies"]["MiddleName"] = "Kumar";
Response.Cookies["StudentCookies"]["LastName"] = "Vatsa";
Response.Cookies["StudentCookies"]["TotalMarks"] = "499";
Response.Cookies["StudentCookies"].Expires = DateTime.Now.AddDays(1);
```

Reading/Getting Cookies

In the above code, I have used many ways to write or create cookies so I need to write here using all the above ways separately.

For Way 1

C# Copy Code

```
string roll = Request.Cookies["StudentCookies"].Value; //For First Way
```

For Way 2

C# Copy Code

```
string roll = Request.Cookies["StudentCookies"].Value;  //For Second Way
```

For Way 3

C# Copy Code

```
//For Multiple values in single cookie
string roll;
roll = Request.Cookies["StudentCookies"]["RollNumber"];
roll = roll + " " + Request.Cookies["StudentCookies"]["FirstName"];
roll = roll + " " + Request.Cookies["StudentCookies"]["MiddleName"];
roll = roll + " " + Request.Cookies["StudentCookies"]["LastName"];
roll = roll + " " + Request.Cookies["StudentCookies"]["TotalMarks"];
Label1.Text = roll;
```

How to create Cookies

For working with cookies, we need to use the namespace `System.web`.

```
using System.Web;
```

Have a look at the code and see how we create cookies and add it with a web response.

```
        //Creting a Cookie Object
        HttpCookie _userInfoCookies = new HttpCookie("UserInfo");
       //Setting values inside it
        _userInfoCookies["UserName"] = "Abhijit";
        _userInfoCookies["UserColor"] = "Red";
        //Adding cookies to current web response
        Response.Cookies.Add(_userInfoCookies);
```

The cookies which have been created will persist until the browser is closed. We can persist cookies beyond that. But

How to read data from Cookies

Now it is time to retrieve data from the cookies. Before reading the cookies, first we need to check whether a cookie was found or not. It is always a good practice to check a cookie before reading it, because the browser might have disabled cookies.

```
        //retrieve cookies by cookies name
        HttpCookie _userInfoCookies = Request.Cookies["UserInfo"];
        string userName;
        //Check cookies
        if (_userInfoCookies != null)
        {
            userName = _userInfoCookies["UserName"];
        }
```

Deleting Cookies

In the above code, I have used many ways to create or read cookies. Now look at the code given below which will delete cookies.

C# Copy Code

```
if (Request.Cookies["StudentCookies"] != null)
{
    Response.Cookies["StudentCookies"].Expires = DateTime.Now.AddDays(-1);
    Response.Redirect("Result.aspx");  //to refresh the page
}
```

How to remove persistent Cookies before its expiration time?

This is a funny task. If you want to remove persistent cookies before the expiration date, the only way is to replace them with cookies with a past expiration date.

C# Copy Code

```
HttpCookie _userInfoCookies = new HttpCookie("UserInfo");
//Adding Expire Time of cookies before existing cookies time
_userInfoCookies.Expires = DateTime.Now.AddDays(-1);
//Adding cookies to current web response
Response.Cookies.Add(_userInfoCookies);
```

5.3 Session Objects - Using session variables

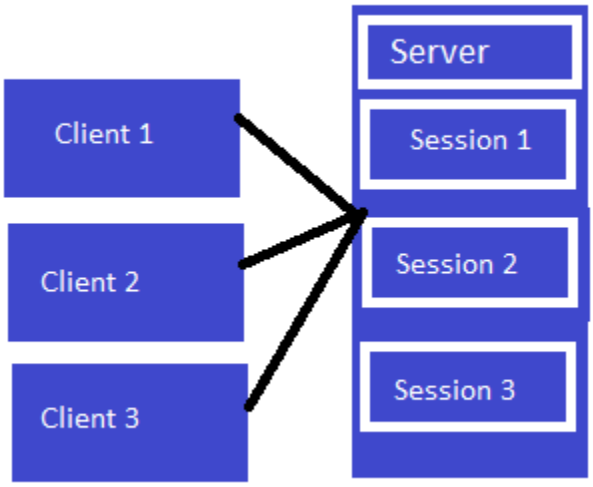
Managing Session State

Session-state management is one of ASP.NET's premiere features. **It allows you to store any type of data in memory on the server**.

The information is protected, because it is never transmitted to the client, and it's uniquely bound to a specific session.

Every client that accesses the application has a different session and a distinct collection of information.

Session state is ideal for storing information such as the items in the current user's shopping basket when the user browses from one page to another.



Session Tracking

ASP.NET tracks each session by using a **unique 120-bit identifier**. ASP.NET uses **a proprietary algorithm to generate this value, thereby guaranteeing (statistically speaking) that the number is unique** and it's random enough that a malicious user can't reverse-engineer or "guess" what session ID a given client will be using. This ID is the only piece of session-related information that is transmitted between the web server and the client.

When the client presents the session ID, ASP.NET looks up the corresponding session, retrieves the objects you stored previously, and places them into a special collection so they can be accessed in your code. This process takes place automatically.

For this system to work, the client must present the appropriate session ID with each request. You can accomplish this in two ways:

- ***Using cookies***: In this case, the session ID is transmitted in a special cookie (named ASP.NET_SessionId), which ASP.NET creates automatically when the session collection is used. This is the default.
- ***Using modified URLs***: In this case, the session ID is transmitted in a specially modified (or *munged*) URL. This allows you to create applications that use session state with clients that don't support cookies.

Ordinarily, ASP.NET uses cookies to track session state. You'll learn how to switch to the modified-URL system (called cookieless sessions)

Using Session State

You can interact with session state by using the **System.Web.SessionState.HttpSessionState** class, which is provided in an ASP.NET web page as the built-in Session object. The syntax for adding items to the collection and retrieving them is basically the same as for adding items to a page's view state.

For example, you might store a DataSet in session memory like this:

```
Session["InfoDataSet"] = dsInfo;
```

You can then retrieve it with an appropriate conversion operation:

```
dsInfo = (DataSet)Session["InfoDataSet"];
```

Of course, before you attempt to use the dsInfo object, you'll need to check that it actually exists—in other words, that it isn't a null reference.

Session state is global to your entire application for the current user. However, session state can be lost in several ways:

- If the user closes and restarts the browser.
- If the user accesses the same page through a different browser window, although the session will still exist if a web page is accessed through the original browser window. Browsers differ on how they handle this situation.
- If the session times out due to inactivity. More information about session timeout can be found in the configuration section.
- If your web page code ends the session by calling the Session.Abandon() method.

In the first two cases, the session actually remains in memory on the web server, because ASP.NET has no idea that the client has closed the browser or changed windows. The session will linger in memory, remaining inaccessible, until it eventually expires. (Ordinarily, that's after 20 minutes)

3.1 Properties of session object :-

Table 8-1 describes the key methods and properties of the HttpSessionState class.

Table 8-1. *HttpSessionState Members*

Member	Description
Count	Provides the number of items in the current session collection.
IsCookieless	Identifies whether the session is tracked with a cookie or modified URLs.
IsNewSession	Identifies whether the session was created only for the current request. If no information is in session state, ASP.NET won't bother to track the session or create a session cookie. Instead, the session will be re-created with every request.
Keys	Gets a collection of all the session keys that are currently being used to store items in the session-state collection.
Mode	Provides an enumerated value that explains how ASP.NET stores session-state information. This storage mode is determined based on the web.config settings.
SessionID	Provides a string with the unique session identifier for the current client.
Timeout	Determines the number of minutes that will elapse before the current session is abandoned, provided that no more requests are received from the client. This

	value can be changed programmatically, letting you make the session collection longer when needed.
Abandon()	Cancels the current session immediately and releases all the memory it occupied. This is a useful technique in a logoff page to ensure that server memory is reclaimed as quickly as possible.
Clear()	Removes all the session items but doesn't change the current session identifier.

A Session-State Example

1. Display information about session object and its properties

```
<%@ Page Language="C#" AutoEventWireup="true" CodeBehind="WebForm4.aspx.cs"
Inherits="Cookies_Session.WebForm4" %>

<!DOCTYPE html>

<html xmlns="http://www.w3.org/1999/xhtml">
<head runat="server">
    <title></title>
</head>
<body>
    <form id="form1" runat="server">
        <asp:Label ID="lblSession" runat="server" Text="Label"></asp:Label>
    <div>

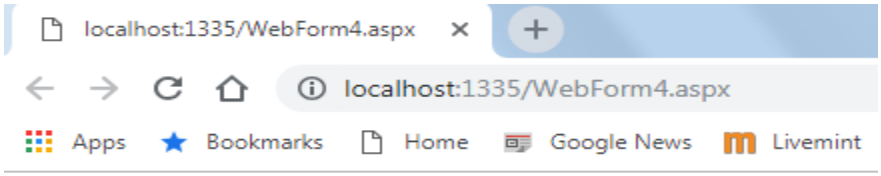
    </div>
    </form>
</body>
</html>
```

```
using System;

namespace Cookies_Session
{
    public partial class WebForm4 : System.Web.UI.Page
    {
        protected void Page_Load(object sender, EventArgs e)
        {
```

```
            // Display some basic information about the session.
            // This is useful for testing configuration settings.
            lblSession.Text = "Session ID: " + Session.SessionID;
            lblSession.Text += "<br />Number of Objects: ";
            lblSession.Text += Session.Count.ToString();
            lblSession.Text += "<br />Mode: " + Session.Mode.ToString();
            lblSession.Text += "<br />Is Cookieless: ";
            lblSession.Text += Session.IsCookieless.ToString();
            lblSession.Text += "<br />Is New: ";
            lblSession.Text += Session.IsNewSession.ToString();
            lblSession.Text += "<br />Timeout (minutes): ";
            lblSession.Text += Session.Timeout.ToString();
        }

        }
    }
```



Session ID: lzbforwpsrbvsg42t2v3q1yq
Number of Objects: 0
Mode: InProc
Is Cookieless: False
Is New: True
Timeout (minutes): 20

2. Passing user information from one webpage to another using session.

WebForm2.aspx

```
using System;

namespace Cookies_Session
{
    public partial class WebForm2 : System.Web.UI.Page
    {
        protected void Page_Load(object sender, EventArgs e)
        {

        }
```

```
        protected void Button1_Click(object sender, EventArgs e)
           {
               Session["name"] = TextBox1.Text;
               Response.Redirect("WebForm3.aspx");
           }
       }

   }
```

WebForm3.aspx

```
using System;
using System.Web.UI;

namespace Cookies_Session
{
    public partial class WebForm3 : System.Web.UI.Page
    {
        protected void Page_Load(object sender, EventArgs e)
        {

            if(!Page.IsPostBack)
            {
                if (Session["name"] != null)
                {
                Label1.Text ="UserName:- " +
Session["name"].ToString();
                }
            }
        }
    }

   }
```

5.4 Application Objects - Using application variables

The ASP.NET application is the collection of all web pages, code and other files within a single virtual directory on a web server. When information is stored in application state, it is available to all the users.

To provide for the use of application state, ASP.NET creates an application state object for each application from the HTTP ApplicationState class and stores this object in server memory. This object is represented by class file global.asax.

Application State is mostly used to store hit counters and other statistical data, global application data like tax rate, discount rate etc. and to keep the track of users visiting the site.

The HttpApplicationState class has the following properties:

Properties	Description
Item(name)	The value of the application state item with the specified name. This is the default property of the HttpApplicationState class.
Count	The number of items in the application state collection.

The HttpApplicationState class has the following methods:

Methods	Description
Add(name, value)	Adds an item to the application state collection.
Clear	Removes all the items from the application state collection.
Remove(name)	Removes the specified item from the application state collection.
RemoveAll	Removes all objects from an HttpApplicationState collection.
RemoveAt	Removes an HttpApplicationState object from a collection by index.
Lock()	Locks the application state collection so only the current user can access it.
Unlock()	Unlocks the application state collection so all the users can access it.

Application state data is generally maintained by writing handlers for the events:

- Application_Start
- Application_End
- Application_Error
- Session_Start

- Session_End

The following code snippet shows the basic syntax for storing application state information:

```
Void Application_Start(object sender, EventArgs e)

{

   Application["startMessage"] = "The application has started.";

}


Void Application_End(object sender, EventArgs e)

{

   Application["endtMessage"] = "The application has ended.";

}
```

How To Read Values from Application State

So to read from Application state is fairly simple. We should just have a safety check to see whether the value we are accessing is `null`, if the data will not be in Application state is not there then it will return `null` and if we'll try to cast it in any different type, it'll throw an exception. Application state stores the data in object form so we need to typecast after reading it. So we can read the value as:

```
if(Application["Message"] !=null)
{
      string message = Application["Message"].ToString();
}
```

- **Example of application object -----number of request received for application**

For example, you could create a global.asax event handler that tracks the number of sessions that have been created or the number of requests that have been received into the application using page load. Here's an example of the latter:

```
protected void Page_Load(Object sender, EventArgs e)
{
    // Retrieve the current counter value.
    int count = 0;
    if (Application["HitCounterForOrderPage"] != null)
    {
```

```
        count = (int)Application["HitCounterForOrderPage"];
    }
    // Increment the counter.
    count++;
    // Store the current counter value.
    Application["HitCounterForOrderPage"] = count;
    lblCounter.Text = count.ToString();
}
```

Once again, application-state items are stored as objects, so you need to cast them when you retrieve them from the collection. Items in application state never time out. They last until the application or server is restarted or the application domain refreshes itself (because of automatic process recycling settings or an update to one of the pages or components in the application). Application state isn't often used, because it's generally inefficient.The counter would probably not keep an accurate count, particularly in times of heavy traffic. For example, if two clients requested the page at the same time, you could have a sequence of events like this:

1. User A retrieves the current count (432).
2. User B retrieves the current count (432).
3. User A sets the current count to 433.
4. User B sets the current count to 433.

In other words, one request isn't counted because two clients access the counter at the same time. To prevent this problem, you need to use the Lock() and Unlock() methods, which explicitly allow only one client to access the Application state collection at a time.

```
protected void Page_Load(Object sender, EventArgs e)
{
    // Acquire exclusive access.
    Application.Lock();
    int count = 0;
    if (Application["HitCounterForOrderPage"] != null)
    {
        count = (int)Application["HitCounterForOrderPage"];
    }
    count++;
    Application["HitCounterForOrderPage"] = count;
    // Release exclusive access.
    Application.Unlock();
    lblCounter.Text = count.ToString();
}
```

Unfortunately, all other clients requesting the page will be stalled until the Application collection is released. This can drastically reduce performance. Generally, frequently modified values are poor candidates for application state. In fact, application state is rarely used in the .NET world.

5.5 Initializing Application and Session variables

5.6 Creating a global.asa file

What is Global.asax file: The Global.asax file, also known as the ASP.NET application file, is an optional file that contains code for responding to application-level events raised by ASP.NET or by HttpModules.

The Global.asax file resides in the root directory of an ASP.NET-based application. The Global.asax file is parsed and dynamically compiled by ASP.NET.

The Global.asax file itself is configured so that any direct URL request for it is automatically rejected; external users cannot download or view the code written within it.

The Global.asax file does not need recompilation if no changes have been made to it. There can be only one Global.asax file per application and it should be located in the application's root directory only.
The Global.asax contains two types of events those are

Events which are fired for every request

Events which are not fired for every request

global.asax allows us to write event handlers that react to global events in web applications.

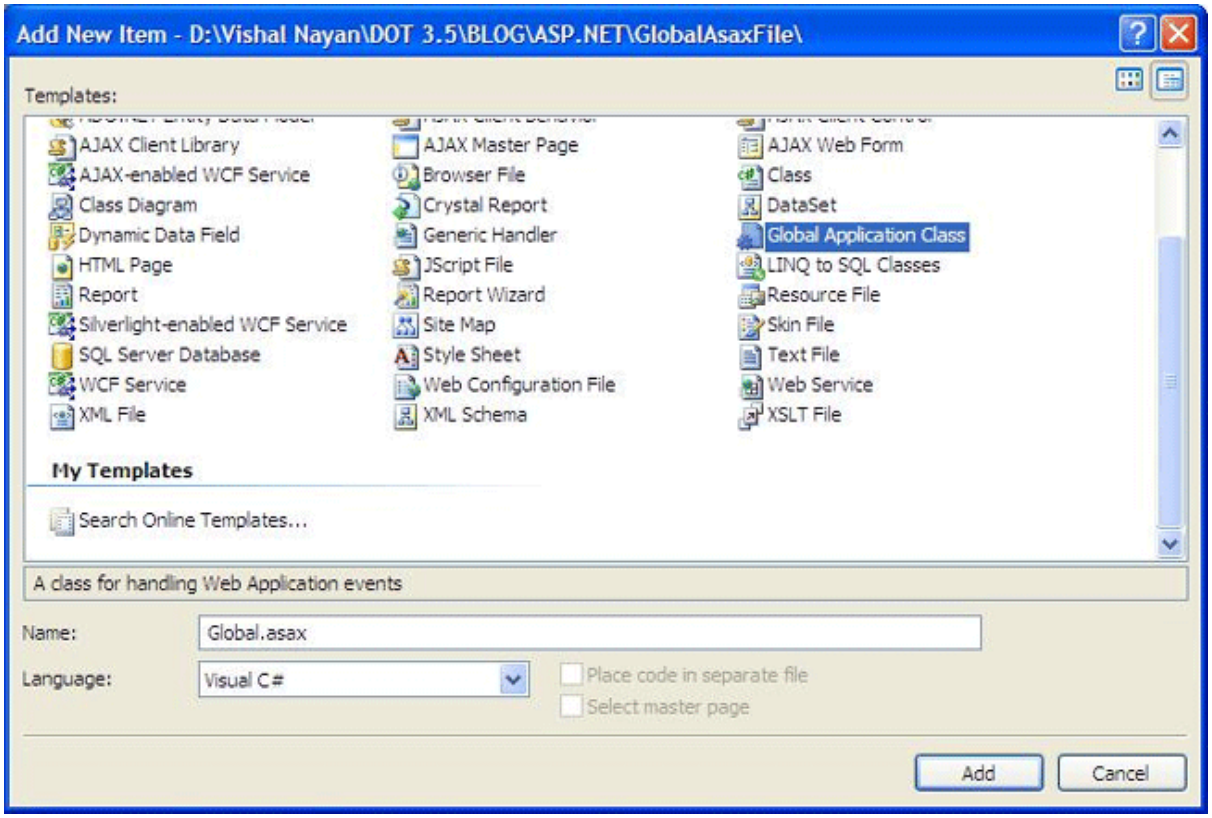
How it gets called: Global.asax files are never called directly by the user, rather they are called automatically in response to application events.

Point to remember about global.asax:

1. They do not contain any HTML or ASP.NET tags.
2. Contain methods with specific predefined names.
3. They defined methods for a single class, application class.
4. They are optional, but a web application has no more than one global.asax file.

How to add global.asax file:

Select Website >>Add New Item (or Project >> Add New Item if you're using the Visual Studio web project model) and choose the Global Application Class template.



After you have added the global.asax file, you will find that Visual Studio has added Application Event handlers:

```
<%@ Application Language="C#" %>
<script runat="server">
    void Application_Start(object sender, EventArgs e)
    {
        // Code that runs on application startup
    }
    void Application_End(object sender, EventArgs e)
    {
        //  Code that runs on application shutdown
    }
    void Application_Error(object sender, EventArgs e)
    {
        // Code that runs when an unhandled error occurs
    }
    void Session_Start(object sender, EventArgs e)
    {
        // Code that runs when a new session is started
    }
    void Session_End(object sender, EventArgs e)
    {
        // Code that runs when a session ends.
        // Note: The Session_End event is raised only when the sessionstate mode
```

```
        // is set to InProc in the Web.config file. If session mode is set to StateServer
        // or SQLServer, the event is not raised.
    }
</script>
```

Purpose of these application event handlers: To reach and handle any HttpApplication event.

If you want to know about HttpApplication events.

What kind of application event global.asax can handle:

Global.asax can handle 2 types of applications events

1) Events that occurs only under specific conditions, i.e. request / response related events.

Order in which application event handler executes;

Order	Application Events	What they do
1	Application_BeginRequest()	This method is called at the start of every request.
2	Application_AuthenticateRequest()	This method is called just before authentication is performed. This is time and place where we can write out own authentication codes.
3	Application_AuthorizeRequest()	After the user is authenticated (identified), it's time to determine the user's permissions. Here we can assign user with special privileges
4	Application_ResolveRequestCache()	This method is commonly used in conjunction with output caching.
5	Application_AcquireRequestState()	This method is called just before session-specific information is retrieved for the client and used to populate the Session collection.
6	Application_PreRequestHandlerExecute()	This method is called before the appropriate HTTP handler executes the request.
7	Application_PostRequestHandlerExecute()	This method is called just after the request is handled.
8	Application_ReleaseRequestState()	This method is called when the session-specific information is about to be serialized from the Session collection so that it's available for the next request.

9	Application_UpdateRequestCache()	This method is called just before information is added to the output cache.
10	Application_EndRequest()	This method is called at the end of the request, just before the objects are released and reclaimed. It's a suitable point for cleanup code.

2) Events that don't get fired with every request.

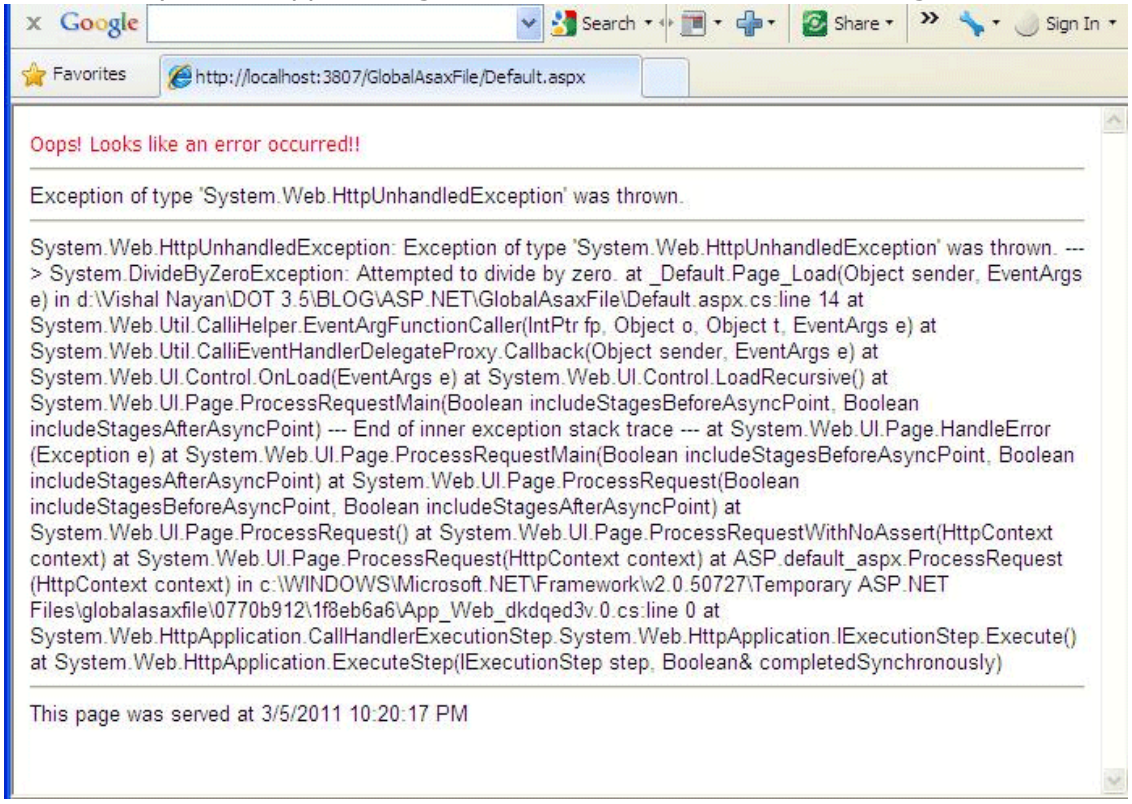
Order	Application Events	What they do
1	Application_Start()	This method is invoked when the application first starts up and the application domain is created. This event handler is a useful place to provide application-wide initialization code. For example, at this point you might load and cache data that will not change throughout the lifetime of an application, such as navigation trees, static product catalogs, and so on.
2	Session_Start()	This method is invoked each time a new session begins. This is often used to initialize user-specific information.
3	Application_Error()	This method is invoked whenever an unhandled exception occurs in the application.
4	Session_End()	This method is invoked whenever the user's session ends. A session ends when your code explicitly releases it or when it times out after there have been no more requests received within a given timeout period (typically 20 minutes).
5	Application_End()	This method is invoked just before an application ends. The end of an application can occur because IIS is being restarted or because the application is transitioning to a new application domain in response to updated files or the process recycling settings.
6	Application_Disposed()	This method is invoked some time after the application has been shut down and the .NET garbage collector is about to reclaim the memory it occupies. This point is too late to perform critical cleanup, but you can use it as a last-ditch failsafe to verify that critical resources are released.

So now it's time to write codes under these event handlers.

Let's try for Application_Error

```
protected void Application_Error(Object sender, EventArgs e)
{
    Response.Write("<font face=\"Tahoma\" size=\"2\" color=\"red\">");
    Response.Write("Oops! Looks like an error occurred!!<hr></font>");
    Response.Write("<font face=\"Arial\" size=\"2\">");
    Response.Write(Server.GetLastError().Message.ToString());
    Response.Write("<hr>" + Server.GetLastError().ToString());
    Server.ClearError();
}
```

So when any error happens, it get handled here with custom message.



5.7 Server object- Methods- CreateObject, Execute, HTMLEncode, MapPath, Transfer

The ASP.NET Server object is used to access properties and methods on the server

The Server object in Asp.NET is an instance of the **System.Web.HttpServerUtility** class.

The HttpServerUtility class provides numerous properties and methods to perform various jobs.

Properties and Methods of the Server object

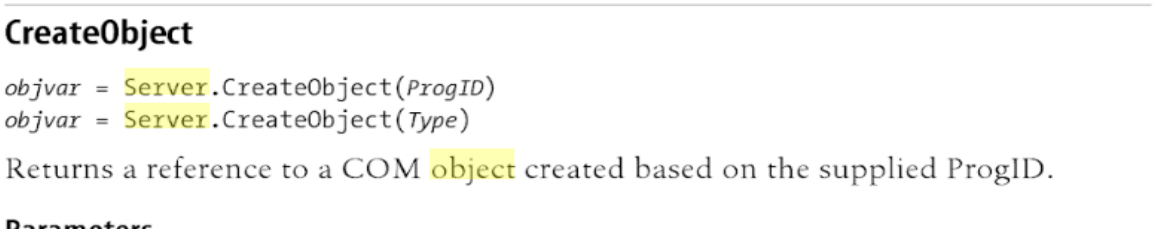
The methods and properties of the HttpServerUtility class are exposed through the intrinsic Server object provided by ASP.NET.

The following table provides a list of the properties:

Property	Description
MachineName	Name of server computer
ScriptTimeOut	Gets and sets the request time-out value in seconds.

Method	Description
CreateObject (String)	Creates an instance of the COM object identified by its ProgID (Programmatic ID).
CreateObject (Type)	Creates an instance of the COM object identified by its Type.
Execute(String)	Executes the handler for the specified virtual path in the context of the current request.

Execute(String, Boolean)	dler for the specified virtual path in the context of the current request and specifies whether to clear the QueryString and Form collections.	
HtmlEncode	Changes an ordinary string into a string with legal HTML characters.	
HtmlDecode	Converts an Html string into an ordinary string.	
MapPath	Return the physical path that corresponds to a specified virtual file path on the server.	
Transfer	Transfers execution to another web page in the current application.	Tra nsf er Transfers execution to another web page in the current application.



You cannot create an instance of an object with the same name as a built-in object:

```
Server.CreateObject("Application")
```

Server.Execute Method

The Execute method calls an .asp file, and processes it as if it were part of the calling ASP script. The Execute method is similar to a procedure call in many programming languages.

Copy

```
Execute(
	Path
)
```

Parameters

- Path
  A string specifying the location of the .asp file to execute. The Path parameter may be for either an absolute or a relative path. If Path is absolute, it must map to an ASP script in the same application as the calling .asp file. Path can be a string variable name that is set at run-time. The Path parameter must not contain a query string, or IIS returns an error.

Transfer and MapPath explanation with examples:

The Transfer() method is the quickest way to redirect the user to another page in your application. When you use this method, a round-trip is not involved. Instead, the ASP.NET engine simply loads the new page and begins processing it. As a result, the URL that's displayed in the client's browser won't change.

```
// You can transfer to a file in the current web application.
Server.Transfer("newpage.aspx");

// You can't redirect to another website.
// This attempt will cause an error.
Server.Transfer("http://www.prosetech.com");
```

The MapPath() method is another useful method of the Server object. For example, imagine you want to load a file named info.txt from the current virtual directory. Instead of hard-coding the path, you can use Server.MapPath() to convert the relative path to your web application into a full physical path. Here's an example:

```
string physicalPath = Server.MapPath("info.txt");

// Now open the file.
StreamReader reader = new StreamReader(physicalPath);
// (Process the file here.)
reader.Close();
```

HTML ENCODE :-

HTML encoding ensures that text will be correctly displayed in the browser, not interpreted by the browser as HTML.

For example, if a text string contains a less than sign (<) or greater than sign (>), the HTML browser would interpret these characters as an opening or closing bracket of an HTML tag.

The HTML encoding of these two characters is `<` and `>`, respectively, which causes the browser to display the less than sign and greater than sign correctly.

```
String text = "you & me > them"; // 1


// Replace > with >
String htmlEncoded = Server.HtmlEncode(text);
```

```
Output:

Step 1: Before encoding has occurred.
String: you & me > them

Step 2: The string is encoded for HTML.
String: you &amp;amp; me &amp;gt; them
```

5.8 Adding web.config file. Creating Simple Master Pages

1.What is Web.Config File?

Configuration file is used to manage ***various settings*** that define a website.

- ❏ The settings are stored in XML files that are **separate from your application code**.
- ❏ In this way you can configure settings independently from your code.
- ❏ Generally a website contains ***a single Web.config file*** stored inside the application root directory.
- ❏ However there can be **many configuration files that manage settings at various levels within an application**.

2. Usage of configuration file

- ❏ It is used to describe ***the properties and behaviors*** of various aspects of ASP.NET applications.
- ❏ Used to ***manage the many settings*** related to your website.

- Each file is an ***XML file*** (with the extension .config) that ***contains a set of configuration elements***.

3. Benefits of XML-based Configuration files

- It can be **stored and retrieved easily**. It is **human readable**.
- You ***need not restart the web server when the settings are changed in the configuration file***. ASP.NET automatically detects the changes and applies them to the running ASP.NET application.
- You can **use any standard text editor or XML parser to create and edit** ASP.NET configuration files.

4. What Web.config file contains?

There are number of important settings that can be stored in the configuration file. Some of the most frequently used configurations, stored conveniently inside Web.config file are:

- **Database connections**
- **Caching settings**
- **Session States**
- **Error Handling**
- **Security**

```
<configuration>

    <connectionStrings>

        <add name="myCon" connectionString="server=MyServer;database=gpkp;uid=xyz;pwd=mydata1223" />

  </connectionStrings>

</configuration/>
```

❑ Web.config Settings

Before we start working with configuration settings of ASP.NET, we see the hierarchy of the *Web.config* file.

Hide Copy Code

```
<configuration>

        <configSections>
            <sectionGroup>
            </sectionGroup>
        </configSections>

        <system.web>
        </system.web>

        <connectionStrings>
        </connectionStrings>

        <appSettings>
        </appSettings>

.....
...

.....
...

.....
...

.....
...

.....
...

</configuration>
```

So from the above tree structure, we can understand that ***the configuration tag is the root element*** of the *Web.config* file under which it has all the remaining sub elements.

Each element **can have any number of attributes** and child elements which specify the values or settings for the given particular section.

system.web

In the configuration hierarchy, the most common thing we will work with is the *system.web* section. Now we look at some of the child sections of the *system.web* section of *Web.config* file.

Compilation Settings

If you want to enable the debugging mode of the application, you can take Compilation settings as granted for these tasks. The following code is used to achieve the discussed settings:

Hide Copy Code

```
<system.web

       <compilation

                  debug="true" strict="true" explicit="true"
batch="true"

                  optimizeCompilations="true" batchTimeout="900"

                  maxBatchSize="1000" maxBatchGeneratedFileSize="1000"

                  numRecompilesBeforeAppRestart="15"
defaultLanguage="c#"

                  targetFramework="4.0" assemblyPostProcessorType="">
       <assemblies>
              <add assembly="System, Version=1.0.5000.0,
Culture=neutral,
                   PublicKeyToken=b77a5c561934e089"/>
       </assemblies>

</compilation>
</system.web>
```

Under the assemblies element, you are supposed to mention the type, version, culture and public key token of the assembly.

Page Settings

Ok, by this time, we have got familiar with the *Web.config* file and we have seen the settings of Compilation Sections, now we will see the settings of a page. As an ASP.NET application consists of several number of pages, we can set the general settings of a page like sessionstate, viewstate, buffer, etc., as shown below:

Hide Copy Code

```
<pages buffer ="true" styleSheetTheme="" theme ="Acqua"

              masterPageFile ="MasterPage.master"

              enableEventValidation="true">
```

By using the MasterPageFile and theme attributes, we can specify the master page and theme for the pages in web application.

Custom Error Settings

The next section of *Web.config* file, we are going to look around is Custom Error settings, by the name itself it is clear that we can configure the settings for the application level errors in these section. Now we will see the description of the `customErrors` section of the *Web.config* from the below mentioned code snippet.

Hide Copy Code

```
<customErrors defaultRedirect ="Error.aspx" mode ="Off">
   <error statusCode ="401" redirect ="Unauthorized.aspx"/>
</customErrors>
```

The customErrors section consists of defaultRedirect and mode attributes which specify the default redirect page and the on/off mode respectively. The subsection of customErrors section allows redirecting to specified page depending on the error status code.

- 400 Bad Request
- 401 Unauthorized

- 404 Not Found
- 408 Request Timeout

Location Settings

If you are working with a major project, probably you might have numerous numbers of folders and sub-folders, at this kind of particular situation, you can have two options to work with.

First thing is to have a *Web.config* file for each and every folder(s) and Sub-folder(s) and the second one is to have a single *Web.config* for your entire application.

If you use the first approach, then you might be in a smoother way, but what if you have a single *Web.config* and you need to configure the sub-folder or other folder of your application, the right solution is to use the "Location" tag of "system.web" section of *Web.config* file. However you can use this tag in either of the discussed methods.

The following code shows you to work with Location settings:

Hide Copy Code

```
<location path="Login.aspx">
       <system.web>
              <authorization>
                <allow users="*"/>
              </authorization>
       </system.web>
</location>
```

Hide Copy Code

```
<location path ="Uploads">
    <system.web>
       <compilation debug = "false">
    </system.web>
</location>
```

In a similar way, you can configure any kind of available settings for any file/folder using the `location` tag.

Session State and View State Settings

As we all know, the ASP.NET is stateless and to maintain the state we need to use the available state management techniques of ASP.NET. View state and session state are among them. For complete information about view state and Session

Now we'll see the *Web.config* settings of View State and Session State:
View State can be enabled or disabled by using the following page settings in the *web.config* file.

Hide Copy Code

```
<Pages EnableViewState="false" />
```

Session state settings for different modes are as shown below:

Hide Copy Code

```
<sessionState mode="InProc" />
```

Hide Copy Code

```
<sessionState mode="StateServer"

stateConnectionString= "tcpip=Yourservername:42424" />
```

Hide Copy Code

```
<sessionState mode="SQLServer" sqlConnectionString="cnn" />
```

Authentication, Authorization Settings

These settings are directly available in the *web.config* file

Authentication Settings

Hide Copy Code

```
<authentication mode="Forms">
```

```
     <forms cookieless="UseCookies" defaultUrl="HomePage.aspx"

                    loginUrl="UnAuthorized.aspx" protection="All"
timeout="30">
      </forms>
</authentication>
```

Authorization Settings

Hide Copy Code

```
<authorization>

        <allow roles ="Admin"/>
        <deny users ="*"/>
</authorization>
```

AppSettings

In the above section, we have seen the settings available in system.web tag,

Now we will see the available settings in appSettings section.
appSettings element helps us to store the application settings information like connection strings, file paths, URLs, port numbers, custom key value pairs, etc.
The following code snippet shows the example of appSettings Section:

Hide Copy Code

```
<appSettings>
       <add key="AppKey" value="APLJI12345AFAFAF89999BDFG"/>
</appSettings>
```

connectionStrings

The most common section of *web.config* file the `connectionStrings` sections allows you to store multiple connection strings that are used in the application. The `connectionStrings` tag

consists of child element with attributes name and `connectionstring` which is used to identify

the `connectionstring` and the other is used to connect to the database server respectively.

The general `connectionstring` settings are shown below:

Hide Copy Code

```
<connectionStrings>
    <add name ="cnn" connectionString ="Initial Catalog = master;
              Data Source =localhost; Integrated Security = true"/>
</connectionStrings>
```

ConfigSections

ConfigSections helps you to create your own custom configuration section that can be used with the *web.config* file.

ConfigSections should be declared just below the configuration (parent element) otherwise it is going through you an error.

Hide Copy Code

```
<configSections>
    <sectionGroup name="pageAppearanceGroup">
      <section

        name="pageAppearance"

        type="PageAppearanceSection"

        allowLocation="true"

        allowDefinition="Everywhere"

      />
 </sectionGroup>
 </configSections>
```

6. Steps for adding web.config file:-

Add -> new item -> Web Configuration File

This looks like as following.

```
<?xml version="1.0"?>
<configuration>

    <system.web>
      <compilation debug="true" targetFramework="4.5.2" />
      <httpRuntime targetFramework="4.5.2" />
    </system.web>

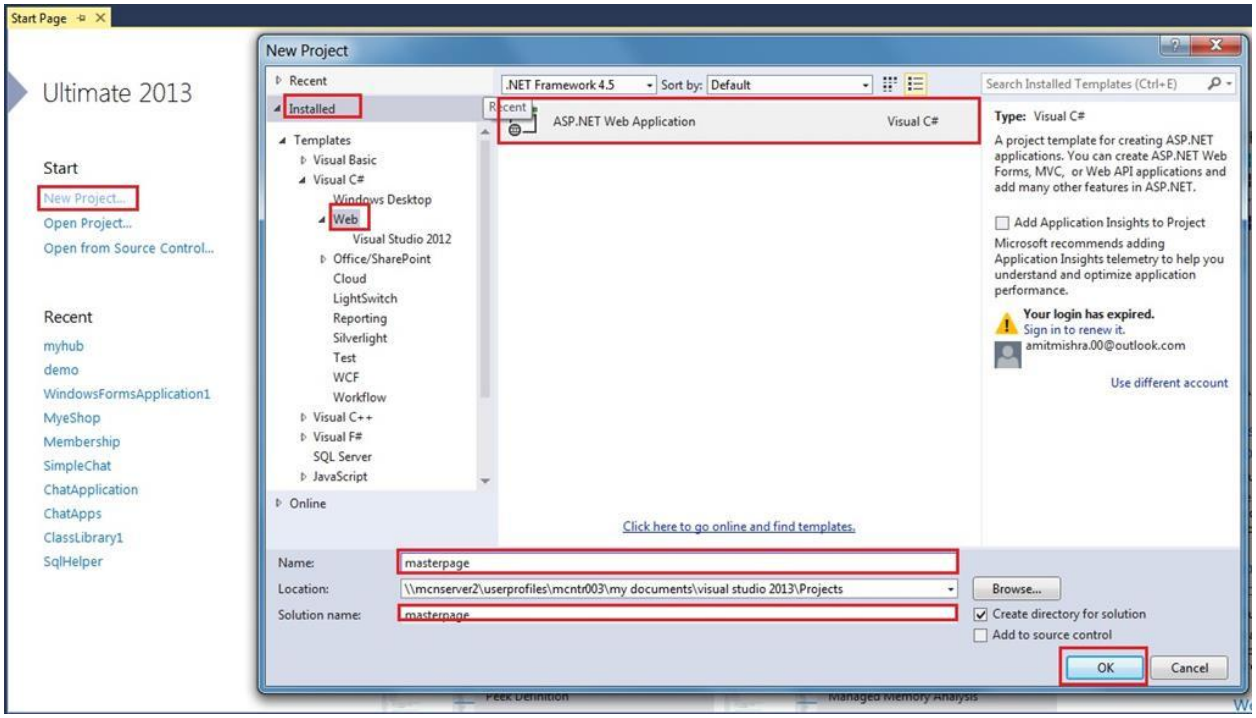
</configuration>
```

Creating Simple Master Pages

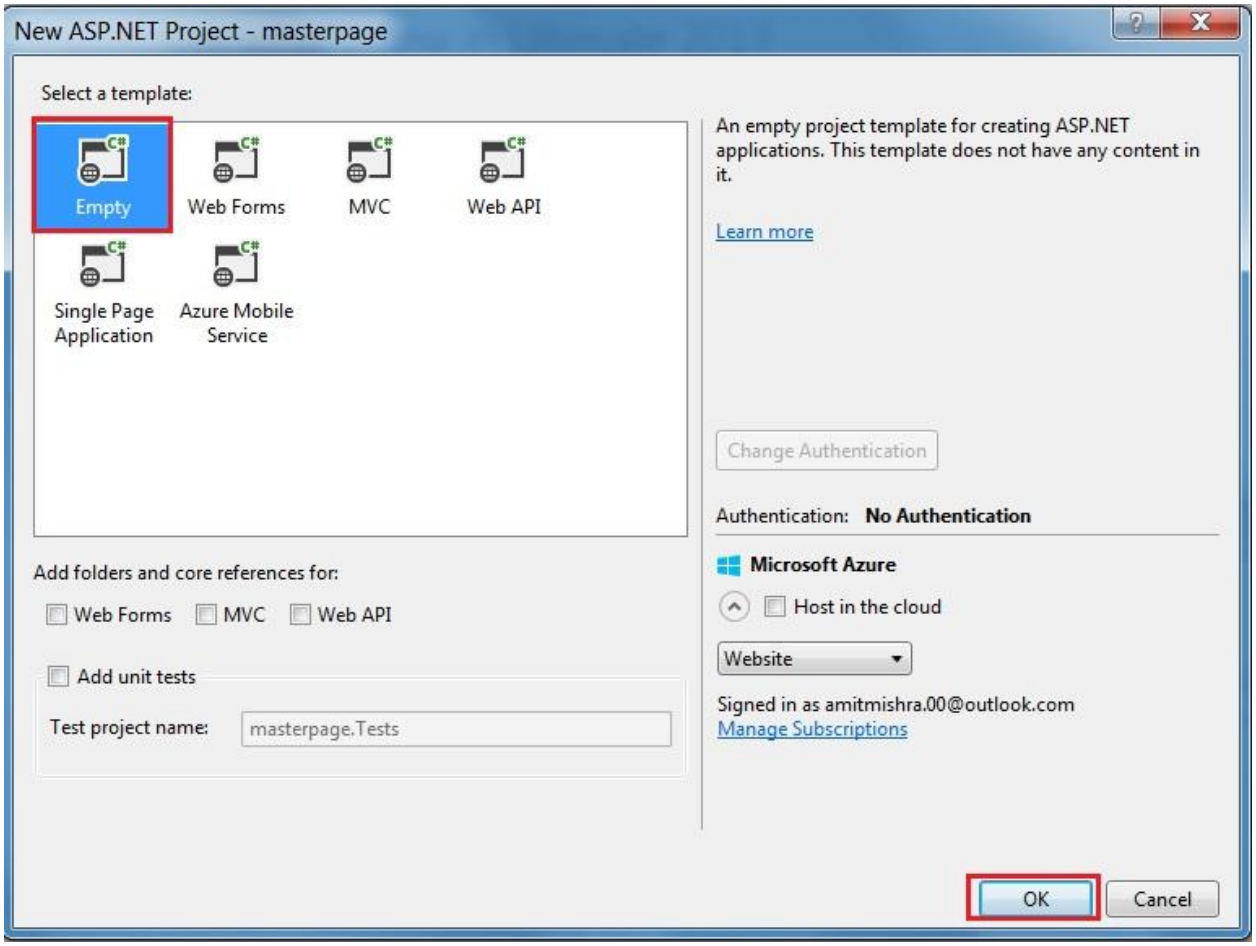
A master page provides the layout and functionality to other pages. Creating a master page in ASP.NET is very easy. Let's start creating master page step by step.

Step 1: Open new project in visual studio

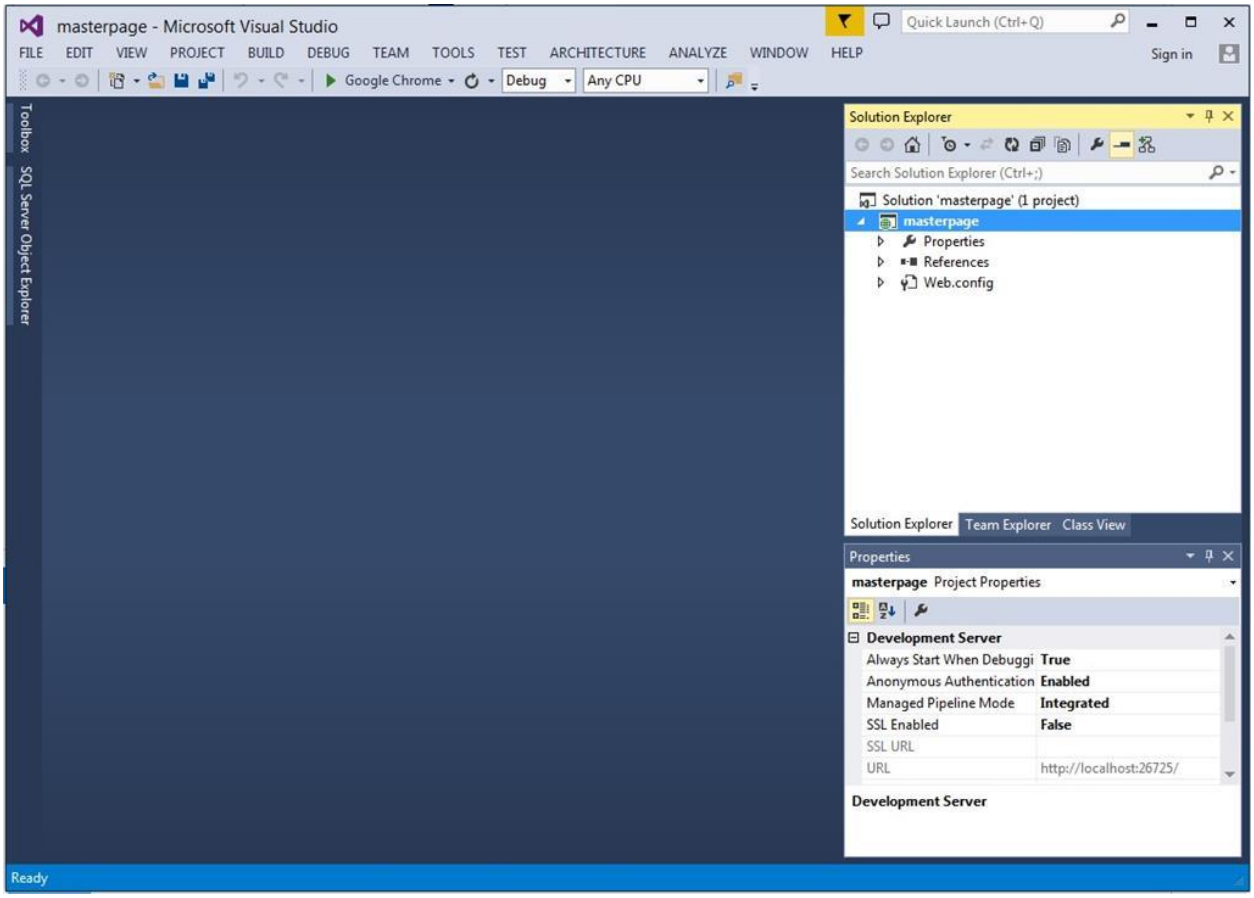
New project->Installed->Web->ASP.NET Web Application (shown in the picture),



After clicking OK button in the Window, select Empty (shown in the picture),



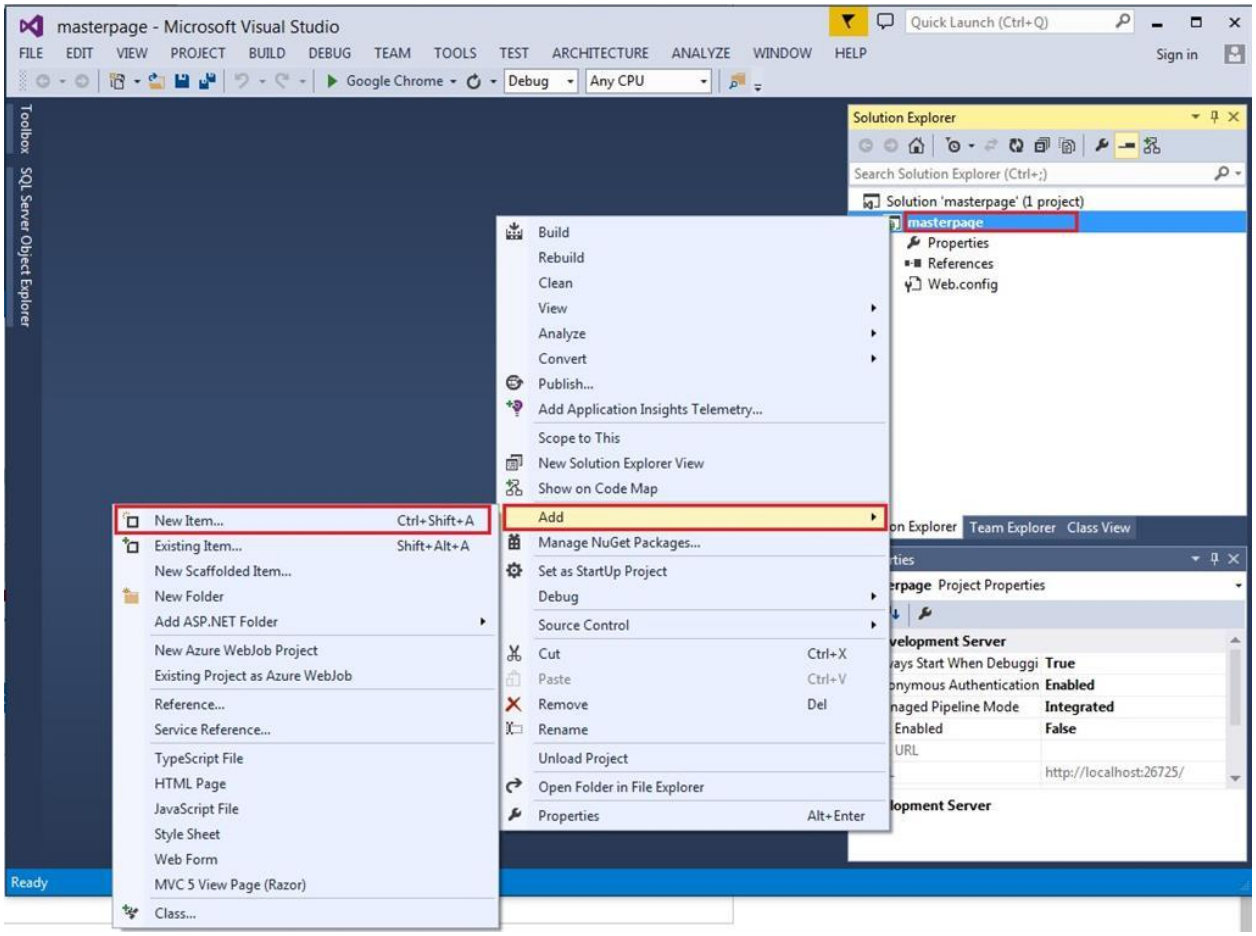
After clicking OK button, project "masterpage" opens but no file is there (shown in the picture),



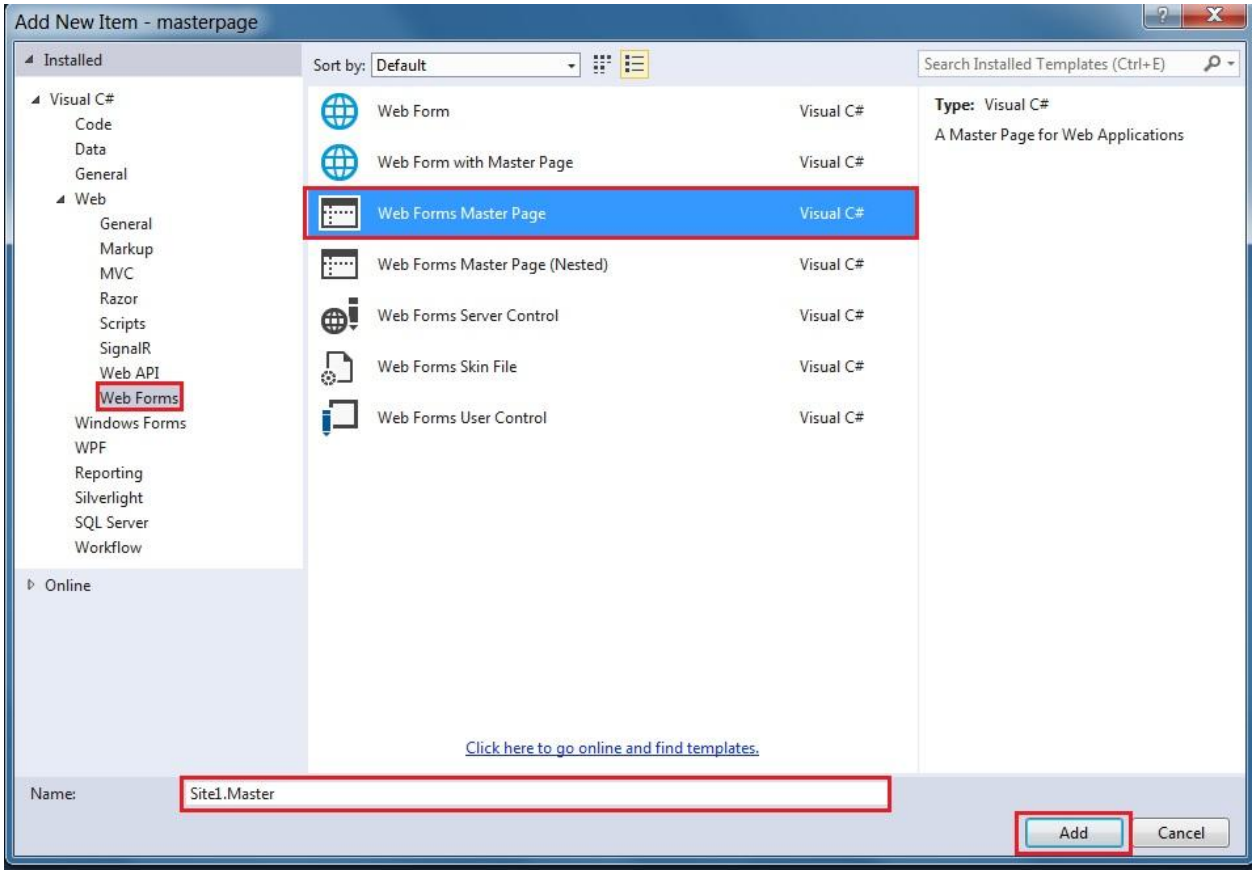
Step 2: Add new file in to our project.

Add the master page into our project.

Right click Project->Add->New item (shown in the picture),

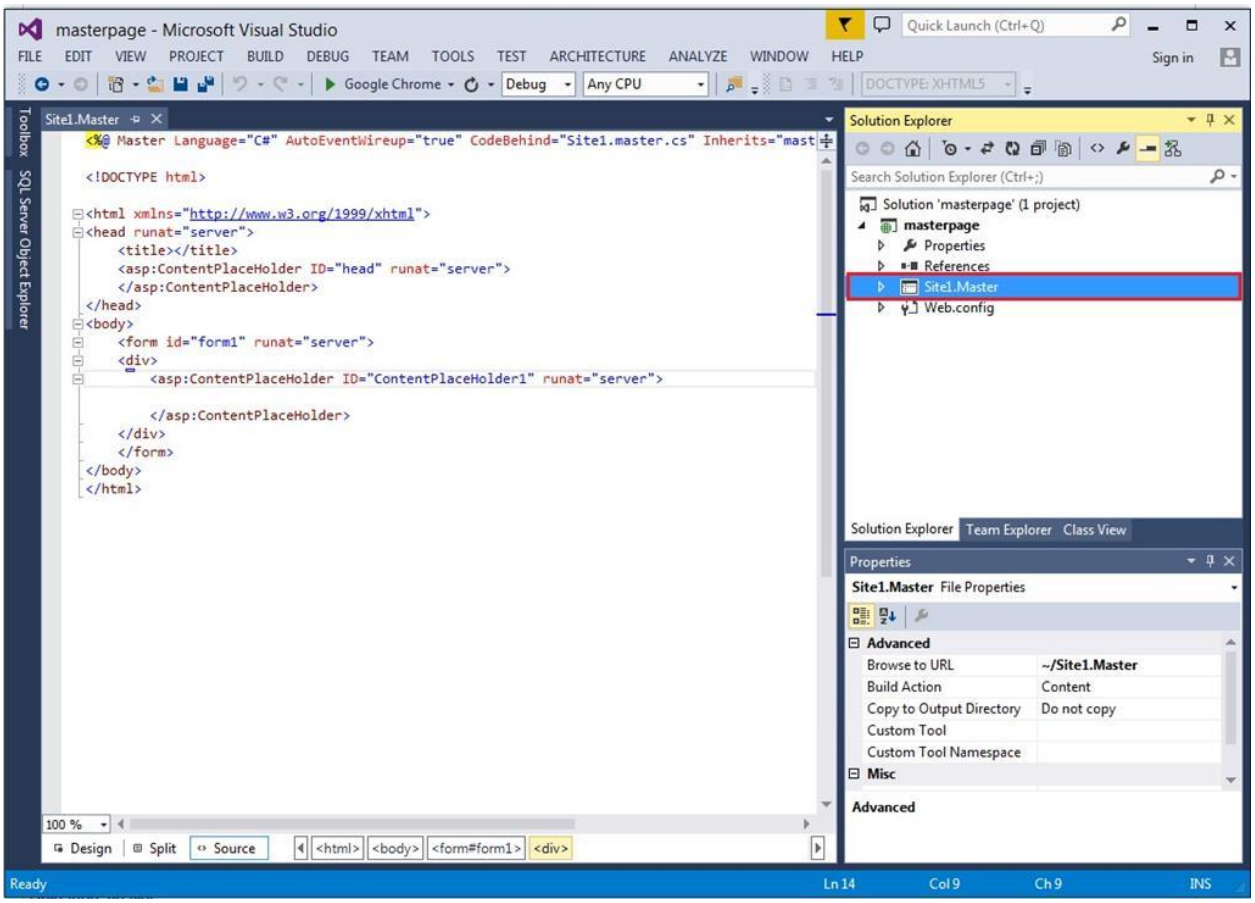


After clicking on new item, Window will open, select Web Form->Web Forms Master Page (shown in the picture),



After clicking the add button, master page 'site1.master' adds to our project.

Click on site1.master into Solution Explorer (shown in the picture),



Step 3: Design the master page, using HTML.

HTML code of my master page is,

```
<%@ Master Language="C#" AutoEventWireup="true"
CodeBehind="Site1.master.cs" Inherits="masterpage.Site1" %>

<!DOCTYPE html>

<html xmlns="http://www.w3.org/1999/xhtml">
<head runat="server">
    <title>c# corner</title>
    <link href="css/my.css" rel="stylesheet" />
    <asp:ContentPlaceHolder ID="head" runat="server">
    </asp:ContentPlaceHolder>
</head>
<body>
    <!DOCTYPE html>
<html>
```

```
<head>
    <title>my layout</title>
    <link rel="stylesheet" type="text/css" href="my.css">
</head>
<body>
<header id="header">
<h1>c# corner</h1>
</header>
<nav id="nav">
    <ul>
        <li><a href="home.aspx">Home</a></li>
        <li><a href="#">About</a></li>
        <li><a href="#">Article</a></li>
        <li><a href="#">Contact</a></li>
    </ul>
</nav>
<aside id="side">
    <h1>news</h1>
    <a href="#"><p>creating html website</p></a>
    <a href="#"><p>learn css</p></a>
    <a href="#">learn c#</a>
</aside>


    <div id="con">
        <asp:ContentPlaceHolder ID="ContentPlaceHolder1"
runat="server">

        </asp:ContentPlaceHolder>
    </div>


<footer id="footer">
    copyright @c# corner
</footer>
</body>
</html>
    <form id="form1" runat="server">

    </form>
```

```
</body>
</html>
```

CSS Code

```
#header{
    color: #247BA0;
    text-align: center;
    font-size: 20px;
}
#nav{
    background-color:#FF1654;
    padding: 5px;
}
ul{

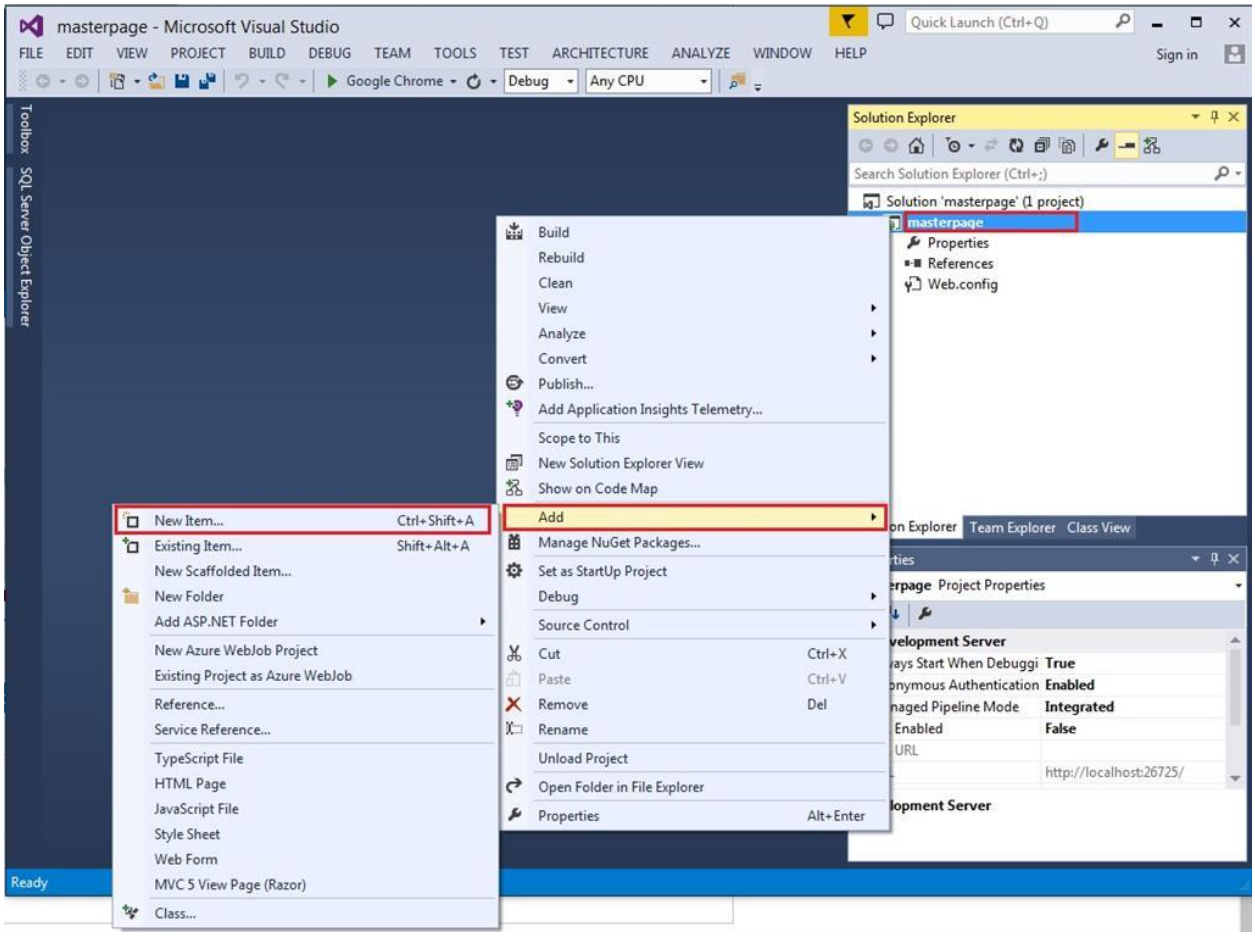
    list-style-type: none;
}
li a {
    color: #F1FAEE;
font-size: 30px;
column-width: 5%;
    }
    li
   {
    display: inline;
    padding-left: 2px;
    column-width: 20px;
   }
   a{
 text-decoration: none;
 margin-left:20px
    }
   li a:hover{
    background-color: #F3FFBD;
    color: #FF1654;
    padding:1%;
   }
   #side{
    text-align: center;
```

```
     float: right;
     width: 15%;
     padding-bottom: 79%;
     background-color: #F1FAEE;
    }
    #article{
     background-color: #EEF5DB;
     padding: 10px;
     padding-bottom: 75%;
    }
    #footer{
     background-color: #C7EFCF;
     text-align:center;
     padding-bottom: 5%;
     font-size: 20px;
}
    #con{
        border:double;
        border-color:burlywood;
    }
```

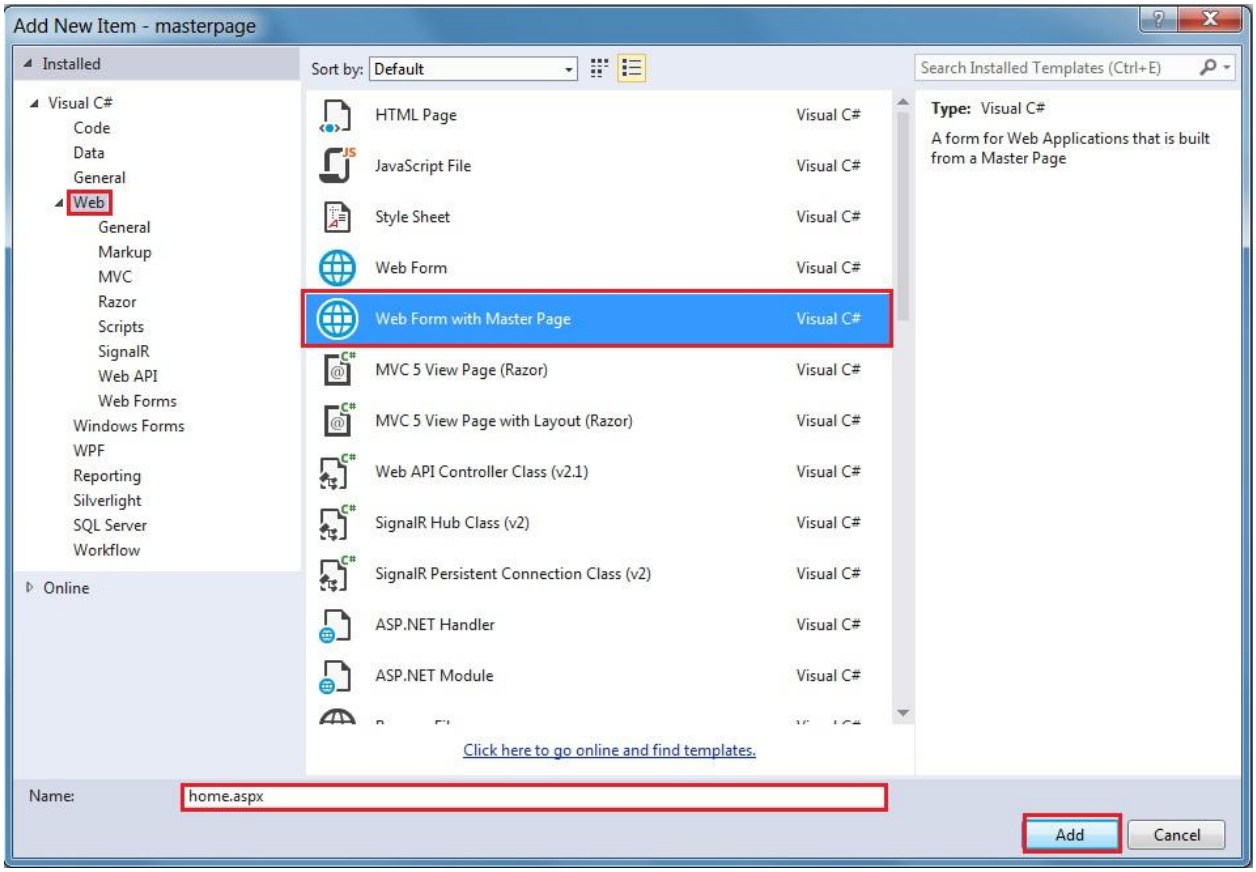
Our master page is designed. Move to the next step.

Step 4: Add web form in to our project.

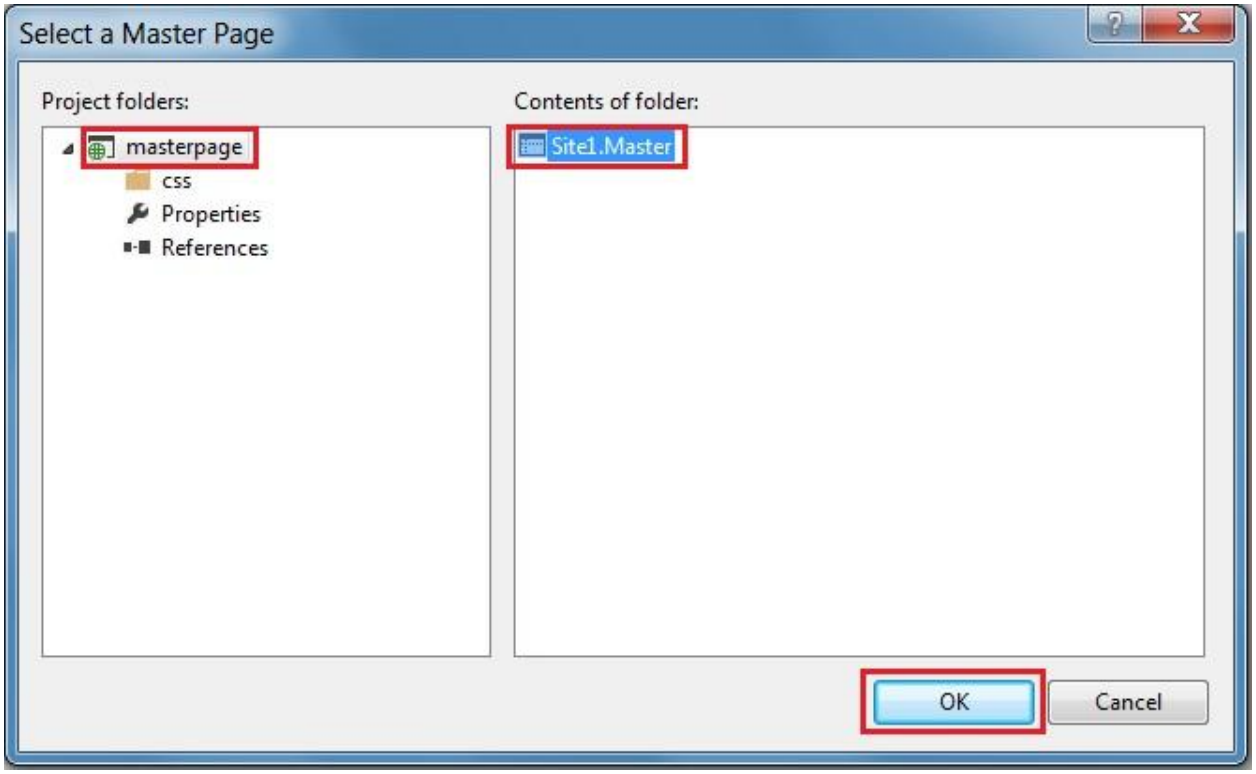
Right click on the project->Add->New item (shown in the picture),



Select Web form with the master page.



After clicking on that, add the button Window, open the selected masterpage->site1.master and click OK.



Now, design our homepage.

Here, we write home page only,

Home.aspx

```
<%@ Page Title="" Language="C#" MasterPageFile="~/Site1.Master" AutoEventWireup="true" CodeBehind="home.aspx.cs" Inherits="masterpage.home" %>
<asp:Content ID="Content1" ContentPlaceHolderID="head" runat="server">
</asp:Content>
<asp:Content ID="Content2" ContentPlaceHolderID="ContentPlaceHolder1" runat="server">
    <h1>Home page</h1>
</asp:Content>
```

Finally, our Master page is created; build and run the project.

The master page looks aas shown in the picture:


localhost:26725/home.aspx
c# corner
Home
About
Article
Contact
Home page
copyright @c# corner
news
creating html website
learn css
learn c#
localhost:26725/home.aspx